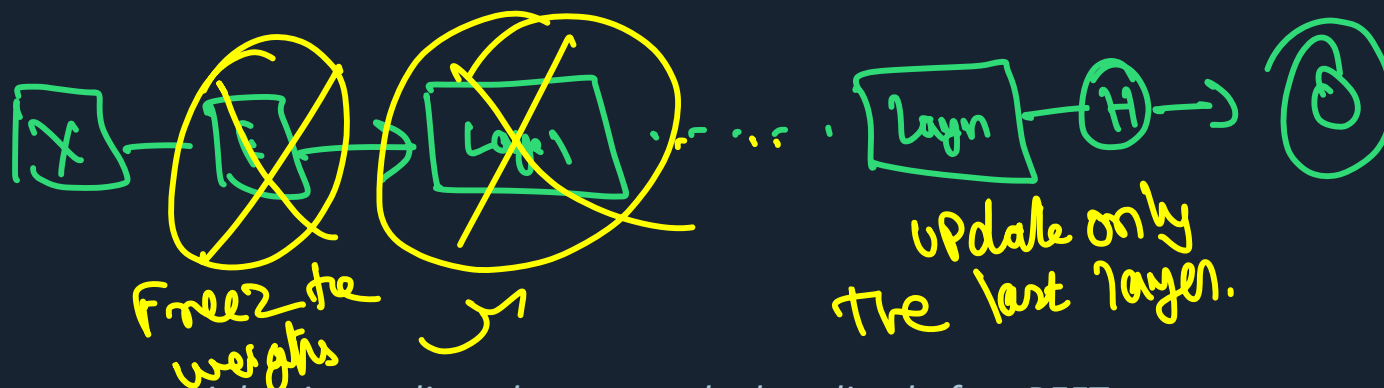


Full-Parameter Fine-Tuning



Updating every weight via gradient descent — the baseline before PEFT

↑
Parametr eff. fine tuning

9 min

Duration

B3 – Apply

Bloom's

L2 – Intermediate

Level

AI-FT-IN-TH-000002

ID

Full-Parameter Fine-Tuning — Definition

Definition

All model parameters are updated using standard gradient descent on a task-specific loss (typically cross-entropy on next-token prediction). Every weight matrix in every layer participates in the backward pass.

Pre-training technique



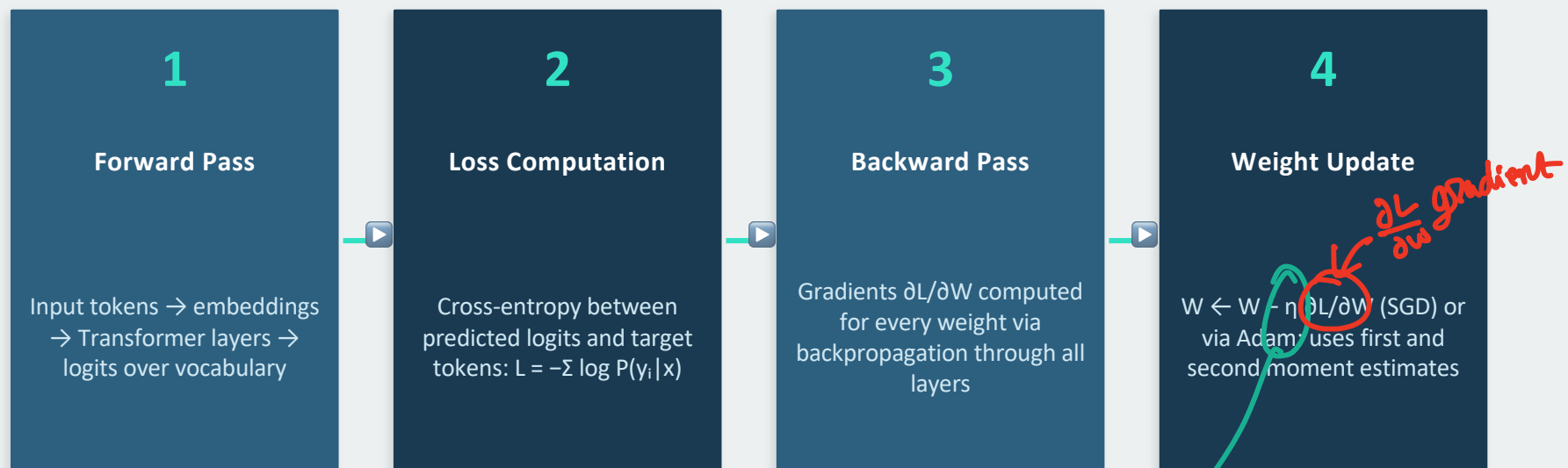
- This is the same algorithm as pre-training — just on a much smaller, curated dataset
- Contrast with PEFT: in full fine-tuning, no weights are frozen
- Result: maximum adaptation — the model can completely shift its behaviour
- Trade-off: maximum compute and memory cost

Challenges:

- no change/update $\approx 0\%$
- Catastrophic Forgetting

The Forward → Loss → Backward → Update Cycle

Full-Fine-tuning (similar to Pre-training)



Memory Anatomy — Why Full FT Needs So Much VRAM

For a 7B-parameter model in mixed precision (bfloat16):

Model weights: $7B \times 2 \text{ bytes} = 14 \text{ GB}$

Gradients: $7B \times 2 \text{ bytes} = 14 \text{ GB}$

Optimizer states (Adam): $7B \times 2 \times 4 \text{ bytes} = 56 \text{ GB}$ (fp32 moments)

Activations (batch): $\sim 8\text{--}28 \text{ GB}$ (sequence-length dependent)

Total $\approx 92\text{--}112 \text{ GB} \rightarrow$ Requires 4–8× A100 80GB GPUs

Consumer GPU (RTX 4090, 24 GB): INFEASIBLE for 7B+ models at full FT

This is exactly why PEFT methods (LoRA, QLoRA) were invented

Indicative

Hyperparameter Guide for Full Fine-Tuning

Hyperparameter	Recommended Range	Effect if Too High	Effect if Too Low
Learning rate	1e-5 to 5e-5	Catastrophic forgetting, instability	Slow convergence, underfitting
Warmup steps	3–10% of total	Skips warmup benefits	Cold start instability
Batch size	16–64 (with grad acc.)	OOM error	Noisy gradients
Gradient accumulation	4–16 steps	Memory savings lost	Effective batch too small
Epochs	1–5	Overfitting to fine-tune data	Underfitting to task
Weight decay	0.01–0.1	Underfitting	Overfitting

I —
I like —
I like sun —

When Is Full Fine-Tuning Justified?

Condition	Full FT?	Reason
Large proprietary dataset (>500K samples)	✓ Yes	Maximum adaptation needed; compute available
Significant domain shift (e.g., base → medical)	✓ Yes	Deep layer adaptation required
Consumer hardware only	✗ No	VRAM infeasible; use QLoRA instead
Small dataset (<10K samples)	✗ No	High overfitting risk; use LoRA
Iterative experiments needed	✗ No	Full FT too slow; PEFT enables fast iteration
Matching a larger model's performance	✓ Sometimes	Full FT on large dataset can close gap

UG in Biology

MBDS

Full Fine-Tuning — Strengths & Limitations

✓ Strengths

- Maximum task performance — all layers adapt
- Best for large-scale domain shift
- No architecture changes needed
- Produces a self-contained checkpoint
- Leveraged by GPT-4, Llama 3, Mistral

⚠ Limitations

- Requires 100+ GB VRAM for 7B+ models
- Catastrophic forgetting risk is highest
- New fine-tune required per task (no composition)
- Slow to iterate: hours–days of training
- One checkpoint per fine-tuned variant (storage cost)

Lab 7.1 Preview — Full Fine-Tuning with HuggingFace Trainer

```
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from datasets import load_dataset

model = AutoModelForCausalLM.from_pretrained("TinyLlama/TinyLlama-1.1B-Chat-v1.0")
tokenizer = AutoTokenizer.from_pretrained("TinyLlama/TinyLlama-1.1B-Chat-v1.0")

dataset = load_dataset("json", data_files="instruction_data.jsonl")

training_args = TrainingArguments(
    output_dir="./full_ft_output",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    learning_rate=2e-5,
    warmup_steps=100,
    logging_steps=50,
    save_strategy="epoch",
    bf16=True,          # mixed precision
)

trainer = Trainer(model=model, args=training_args, train_dataset=dataset["train"])
trainer.train()
model.save_pretrained("./full_ft_checkpoint")
```

This establishes the full FT baseline. All 1.1B parameters are updated. Compare loss curves and GPU memory against LoRA in Lab 7.2.

KEY REFERENCES

- Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
- HuggingFace Transformers Trainer API. huggingface.co/docs/transformers/main_classes/trainer
- Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.



Next: Concept 3: Fine-Tuning vs Pre-Training (AI-FT-IN-TH-000003)